SoleCount: Automated Deployment

After a short one-time setup, a single command (or one button on GitHub) does all of this: **generate the Xcode project → run the tests → take the screenshots → build and sign → upload to TestFlight → upload the App Store text and screenshots → submit for review → release automatically once approved.**

> **Honest limit:** Apple doesn't let any tool do a few steps (your enrollment, agreements, tax and bank details, and adding the *first* In-App Purchase to a review). Each one is a few clicks, done **once**. See "What can never be automated" at the end.

What's in the project

File	What it does
`project.yml`	Describes the Xcode project. XcodeGen turns it into `SoleCount.xcodeproj` (iPhone only, iOS 18, In-App Purchase, icon, StoreKit test file, test targets).
`SoleCountTests/`	Unit tests (Swift Testing) for cost per wear and the 3-pair free limit.
`SoleCountUITests/`	A UI test that adds a pair, a UI test for the free limit, and the screenshot robot.
`SoleCount.storekit`	Lets you test the $0.99 purchase in the Simulator without the App Store.
`fastlane/Fastfile`	The lanes: `setup`, `test`, `screenshots`, `beta`, `release`, `deploy`.
`fastlane/asc_api.rb`	Uses Apple's API for the bundle ID, price, countries, age rating, and the In-App Purchase.
`fastlane/metadata/`	The App Store name, subtitle, description, keywords, URLs, and review notes.
`.github/workflows/ios.yml`	GitHub Actions: tests on every push; TestFlight and App Store when you push a version tag.
`scripts/setup_mac.sh`	One-time Mac setup.
`scripts/deploy.sh`	The single deploy command.

How signing works: Xcode's own *automatic signing* runs with your App Store Connect API key (`-allowProvisioningUpdates` plus the key). Apple creates and stores the distribution certificate in the cloud, so there's no certificate file, no extra private repo, and no password. The key must have the **Admin** role, because that's what cloud signing requires.

One-time setup (about 45 minutes, plus Apple's waiting time)

Do these steps in order. Each one is done once.

Step 1. Join the Apple Developer Program (manual)

1. On your iPhone or Mac, open **https://developer.apple.com/programs/enroll/**, then **Start Your Enrollment**.
2. Sign in with your Apple ID (two-factor authentication must be on), choose **Individual / Sole Proprietor**, and pay **US$99**.
3. Wait for the "Welcome to the Apple Developer Program" email. It usually takes 24–48 hours.

Step 2. Agreements, tax and bank (manual, needed for the $0.99 purchase)

1. Go to **https://appstoreconnect.apple.com**, then **Business** (older accounts: **Agreements, Tax, and Banking**).
2. **Paid Apps Agreement**: click **View and Agree to Terms**, tick the box, then **Agree**.
3. **Bank Account**: click **Add**, enter your bank details, then **Save**.
4. **Tax Forms**: click **Set Up Tax Forms**. Complete the **U.S. Form W-8BEN** (as a Canadian resident, you claim the Canada–U.S. treaty) and the **Canadian** tax form if it's shown.
5. Wait until the status shows **Active**.
6. Team ID: open **https://developer.apple.com/account**, then **Membership details**, and copy the 10-character **Team ID**.

Step 3. Create the App Store Connect API key (manual, 2 minutes)

1. Go to **https://appstoreconnect.apple.com**, then **Users and Access**.
2. Click the **Integrations** tab, then **App Store Connect API** on the left, then **Team Keys**.
3. The first time only, click **Request Access**, tick the box, and click **Submit**. Access is granted right away for Account Holders.
4. Click **Generate API Key** (or the **+** button).
5. **Name:** `SoleCount CI`. **Access: Admin**. Click **Generate**.
6. Click **Download** on the new row. ⚠️ **You can only download it once.** The file is named `AuthKey_XXXXXXXXXX.p8`.
7. Copy the **Key ID** from that row, and the **Issuer ID** shown above the table.
8. **Never** put the `.p8` file in the project folder or on GitHub. Keep it in Downloads. The setup script moves it to a safe place.

Step 4. Run the Mac setup (one command)

In Terminal on your Mac:

```
git clone https://github.com/KrishnaDistributedcomputing/SoleCount.git ~/Desktop/SoleCount
cd ~/Desktop/SoleCount
bash scripts/setup_mac.sh
```

The script: 1. Checks Xcode 26. 2. Installs Homebrew, XcodeGen and fastlane. 3. Asks for your Team ID, Key ID and Issuer ID. 4. Moves the `.p8` key to `~/.appstoreconnect/private_keys/`. 5. Asks for the phone and email App Review should use. 6. Generates and opens `SoleCount.xcodeproj`.

Your answers are saved in `fastlane/.env`. Git ignores that file, so it never goes to GitHub.

Step 5. Create everything in App Store Connect (one command)

```
fastlane setup
```

This registers the bundle ID. It also sets the price to **Free**, all countries, the age rating questionnaire (4+), and **SoleCount Pro** ($0.99, name, description, review note).

One step still needs a login: Apple's API can't create a *new app record*. Choose one: - **A (website, 2 minutes):** go to **App Store Connect** → **Apps** → **+** → **New App**. - Platform **iOS** - Name `SoleCount: Sneaker Tracker` - Language **English (U.S.)** - Bundle ID `com.krishnav.SoleCount` - SKU `solecount-ios` - Full Access

Click **Create**, then run `fastlane setup` again. - **B (Terminal):** add `APPLE_ID=you@example.com` to `fastlane/.env` and run `fastlane setup`. You'll get a single 2FA code prompt. Option B also fills in **App Privacy** ("Data Not Collected") for you.

If you used A, set App Privacy manually: go to **App Privacy** → **Get Started** → **"No, we do not collect data from this app"** → **Save** → **Publish**.

Step 6. GitHub secrets (for the GitHub button, optional)

On your Mac, copy the key as base64:

```
base64 -i ~/.appstoreconnect/private_keys/AuthKey_XXXXXXXXXX.p8 | pbcopy
```

Then, on github.com, go to your **SoleCount repo** → **Settings** → **Secrets and variables** → **Actions** → **New repository secret** and add:

Name	Value
`ASC_KEY_ID`	Key ID (10 characters)
`ASC_ISSUER_ID`	Issuer ID

Name	Value
`ASC_KEY_P8`	Paste (the base64 text you just copied)
`TEAM_ID`	Team ID (optional if it's already in `Config/Team.xcconfig`)

Also commit the files `setup_mac.sh` changed: `Config/Team.xcconfig` and `fastlane/metadata/review_information/*`. They're not secret.

Deploying (every release)

On your Mac, one command:

```
bash scripts/deploy.sh
```

It runs the tests, then uploads to TestFlight, then submits to App Review with **automatic release**. To stop after TestFlight, use `bash scripts/deploy.sh --beta-only`.

Or on GitHub (no Mac needed): - **Push a tag** to run tests, TestFlight, and App Review: `git tag v1.0.0` then `git push origin v1.0.0`. - **Or use the button**: go to **GitHub** → **Actions** → **iOS** → **Run workflow**. Tick **release** to also submit. - Every push to `main` only runs the tests.

The very first release only: Apple requires the **first** In-App Purchase to be added to a review on the website. `release` uploads everything and then shows you this last click: 1. Go to **App Store Connect** → **Apps** → **SoleCount**. 2. In the sidebar, open **Monetization** → **In-App Purchases**, then **SoleCount Pro**, then **Add for Review**. 3. When asked, select iOS app version **1.0**, check the list, then click **Submit for Review**.

Once Pro has been approved, every later release is 100% hands-off.

New version later: change `MARKETING_VERSION` in `project.yml` (for example to `1.1`), write `fastlane/metadata/en-US/release_notes.txt`, then deploy. Build numbers go up automatically.

Troubleshooting

Message	Fix
`Missing ASC_KEY_ID …`	Run `bash scripts/setup_mac.sh` (Mac), or add the GitHub secrets (step 6).
`HTTP 401`	Key ID, Issuer ID or `.p8` don't match. Re-check step 3, then run `bash scripts/setup_mac.sh` again.
`HTTP 403` / "Cloud signing permission error"	The API key must be **Admin**, and all agreements must be accepted (step 2).
`No App Store Connect app … yet`	Create the app record (step 5, option A or B).
`No profiles for 'com.krishnav.SoleCount'` / signing error	Check the Team ID in `Config/Team.xcconfig`. In Xcode, go to **Settings** → **Accounts** and sign in once, then try again.
"Maximum number of certificates"	Each fresh CI machine may create an Apple Development certificate. Go to **developer.apple.com** → **Certificates**, revoke old "Apple Development" ones, then retry.
`Unable to find a destination … iPhone 17 Pro Max`	Open **Xcode** → **Settings** → **Components** and install the iOS Simulator, or set `SIM_DEVICE` in `fastlane/.env` to a simulator you have (`xcrun simctl list devices`).
Bundle ID or app name already taken	Use `com.krishnav.SoleCount2`. Change it in `project.yml`, `fastlane/Appfile` and `fastlane/Fastfile` (`APP_IDENTIFIER`). The product ID can stay.
`Fill in …/phone_number.txt`	Put your phone (with country code) and email in `fastlane/metadata/review_information/`.
Build stuck "Processing"	Wait 5–30 minutes, then run `fastlane release` again.
Age rating asks a NEW question	The setup log lists it. Answer it once on the website (App Information → Age Rating).
Tests fail on GitHub but pass on the Mac	Download the **test-results** artifact from the Actions run. Open the `.xcresult` file in Xcode.

What can be automated and what can't

Task	Automated?	How
Xcode project, icon, capabilities, Info.plist	✅	XcodeGen (`project.yml`)
Unit and UI tests	✅	`fastlane test` and GitHub Actions
6.9" screenshots	✅	`fastlane screenshots` (snapshot)
Certificates, profiles, signing	✅	Xcode cloud signing with the Admin API key
Build numbers, build, TestFlight upload	✅	`fastlane beta`
Bundle ID	✅	API (`asc_api.rb`)
App price (Free), countries	✅	API
Age rating questionnaire	✅	API. New Apple questions may need one manual answer.
In-App Purchase: create, name, $0.99, countries, review screenshot	✅	API
Description, keywords, URLs, screenshots, review notes	✅	`deliver`
Submit for review and automatic release	✅	`deliver` (from the 2nd release on, or once Pro is approved)
App record (first time)	⚠️ one login	Website, or fastlane with your Apple ID (one 2FA prompt). Apple's API doesn't allow it.
App Privacy answers	⚠️ one login	Website, or fastlane with your Apple ID. No API-key endpoint exists.
Adding the **first** In-App Purchase to a review	❌ manual (once)	Apple: the first IAP must be submitted with a version on the website
Developer enrollment, identity check, $99 payment	❌ manual	Apple
Paid Apps Agreement, tax forms, bank	❌ manual	Apple (legal signature)
Creating the API key itself	❌ manual	Apple (it's the key everything else uses)
EU Digital Services Act trader status	❌ manual	App Store Connect → Business. Or set `EXCLUDE_EU=1` until it's done.
Apple's review decision	❌	Apple (usually under 24–48 hours)

Sources: - fastlane, App Store Connect API: https://docs.fastlane.tools/app-store-connect-api/ - fastlane, App Privacy upload: https://docs.fastlane.tools/uploading-app-privacy-details/ - Apple, Apps API (says not to use the API to create new apps): https://developer.apple.com/documentation/appstoreconnectapi/apps - Apple, In-App Purchases API: https://developer.apple.com/documentation/appstoreconnectapi/in-app-purchases - Apple, submitting the first IAP with a version: https://developer.apple.com/help/app-store-connect/manage-submissions-to-app-review/submit-an-in-app-purchase - Apple, Age Rating Declarations API: https://developer.apple.com/documentation/appstoreconnectapi/ageratingdeclarationupdaterequest - Apple, cloud-managed certificates: https://developer.apple.com/help/account/certificates/cloud-managed-certificates/ - XcodeGen project spec: https://github.com/yonaskolb/XcodeGen/blob/master/Docs/ProjectSpec.md - GitHub

macOS 26 runners: https://github.blog/changelog/2026-02-26-macos-26-is-now-generally-available-for-github-hosted-runners/